Hands-On Planning Report
Exploration with Kinodynamic Planning

Houssam Korib, Umut Calsikan, Mostafa Ahmed
Robotics and Autonomous Systems Laboratory
Supervised by: Prof. Narcis Palomeras
Date: June 4, 2026

***Abstract*—This report presents an autonomous exploration stack that integrates frontier-based goal selection, kinodynamic RRT planning, and Pure Pursuit path following with SLAM Toolbox map generation. The planner generates collision-free, dynamically feasible arc trajectories under bounded linear and angular velocities, while the follower tracks these trajectories with speed scaling and in-place reorientation when required. For real deployment, we evaluate two localization variants: (i) a SLAM Toolbox-based baseline, and (ii) a hybrid variant that keeps SLAM Toolbox for `/map` while replacing robot pose input with an optimized `/odom_opt` from a GTSAM+NDT localization pipeline. We explicitly distinguish the simulation baseline from real-world deployment modifications (odometry relay, safety tuning, and anti-stuck frontier handling). The resulting system provides robust exploration behavior in cluttered indoor environments using a modular ROS 2 architecture.**

***Index Terms*—Kinodynamic Planning, Motion Planning, Robot Exploration, LiDAR, Path Planning, Trajectory Control**

I. Introduction

Autonomous mobile robot navigation in unknown environments requires seamless integration of three critical components: real-time environment mapping, motion planning respecting robot constraints, and precise trajectory control. This project addresses these challenges through a unified system combining occupancy-grid mapping with kinodynamic motion planning.

The primary contribution of this work is the development of a smooth and continuous robot navigation system that avoids inefficiencies of conventional advance-stop-turn motion patterns. Rather than commanding discrete primitives, our kinodynamic planner generates trajectories respecting robot dynamics. This design is critical for real-world deployment, as discontinuous motion consumes additional energy, reduces exploration efficiency, and introduces mechanical stress on the robotic platform.

The system integrates frontier-based exploration, kinodynamic RRT planning, odometry fusion, and pure pursuit control in a modular, asynchronous pipeline. Each component operates at optimal frequency while contributing to global exploration objectives. In real experiments, we compare two localization configurations while preserving the same planning and control core: a SLAM Toolbox baseline and a hybrid setup that uses SLAM Toolbox for `/map` and GTSAM+NDT optimized pose for `/odom_opt`.

II. Related Works

Frontier-based exploration remains a standard baseline for autonomous mapping in unknown environments due to its simplicity and strong practical performance [2]. In parallel, kinodynamic sampling-based planners extend geometric planning by explicitly respecting system motion constraints during expansion [1], [6]. For differential-drive robots, this is particularly important in narrow or high-curvature environments where purely geometric shortcuts are infeasible.

Pure Pursuit is a widely used trajectory-tracking method thanks to its low computational cost and stable behavior on piecewise-smooth paths [4]. Modern ROS 2 exploration stacks commonly pair mapping back-ends such as SLAM Toolbox [3] with sampling-based global planning and lightweight local tracking. Our implementation follows this direction, with emphasis on deployment-oriented robustness: heading-aware kinodynamic steering, conservative occupancy-based safety checks, and frontier-goal recovery under repeated planning failures.

III. Methodology

A. System Architecture

The autonomous navigation architecture comprises four primary nodes operating in a continuous feedback loop. The **Hybrid Odometry Relay** merges wheel-odometry twist with SLAM-consistent pose to publish `/odom_opt`. The **Frontier-Based Explorer** identifies and prioritizes unexplored regions. The **Kinodynamic RRT Planner** generates dynamically feasible trajectories under velocity bounds and occupancy constraints. The **Pure Pursuit Controller** executes trajectories via velocity-based path tracking. This modular design supports online replanning while maintaining exploration progress.

B. Kinodynamic RRT Planner

Unlike standard RRT algorithms that generate purely geometric paths, this approach incorporates forward-simulation to ensure every generated edge is dynamically feasible for differential drive robots.

1) Sampling and Tree Growth: The planner initializes the search tree at the robot's current state $\mathbf{q}_0 = (x_0, y_0, \theta_0)$. The expansion process follows an iterative refinement loop:

1) **Adaptive Sampling:** Goal frontier selected with 10% bias; 90% of samples drawn uniformly from accessible occupancy grid.

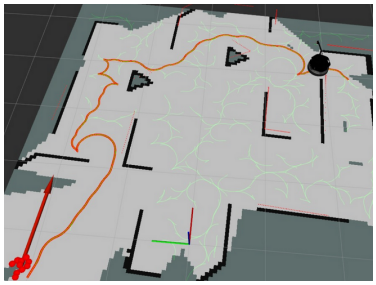


Fig. 1: Visualization of the Kinodynamic RRT tree expansion and selected path in RViz.

2) **State-Aware Nearest Selection:** Tree searched for node N_{near} minimizing a weighted state distance combining sampled-point position error and heading mismatch.
3) **Kinodynamic Motion Primitives:** Planner tests a discrete control set from $A = \{v, \omega\}$, where $v \in \{v_{\max}, 0.75v_{\max}, 0.4v_{\max}, 0\}$ and $\omega \in \{0, \pm\omega_{\max}, \pm 0.5\omega_{\max}\}$.
4) **Forward Dynamics Simulation:** Each action is integrated over horizon $\Delta t = 0.5$ s using 10 internal integration steps:

$$x_{\text{new}} = x + v\cos(\theta) \cdot \Delta t \tag{1}$$
$$y_{\text{new}} = y + v\sin(\theta) \cdot \Delta t \tag{2}$$
$$\theta_{\text{new}} = \theta + \omega \cdot \Delta t \tag{3}$$

5) **Action Selection and Collision Validation:** In high heading-error cases, the steering cost prioritizes heading alignment (allowing in-place rotation); otherwise it prioritizes position progress with a heading term. Candidate arcs are accepted only if collision-free under circular footprint inflation with $r_{\text{robot}} = 0.20$ m.
6) **Termination:** Expansion halts once a state reaches within 0.30 m of goal frontier.

The complete kinodynamic RRT algorithm is presented in Algorithm 1. The algorithm maintains a tree of nodes $\mathcal{V}$ and a parent map $\mathcal{T}$ that stores both parent pointers and the simulated arcs connecting nodes. At each iteration, a random point is sampled (with 10% goal bias), the nearest tree node is identified, and the steering function attempts to extend toward the sampled point using motion primitives. When a collision-free arc is found, the new node is added to the tree. Once a node reaches the goal region, the path is reconstructed by backtracking through the parent map and concatenating the stored arcs, yielding a smooth, kinodynamically-feasible trajectory.

Algorithm 1 Kinodynamic RRT

Require: x_{start}, x_{goal}, bounds, limits
Ensure: Path $\mathcal{P}$ or None, parent map $\mathcal{T}$

if $\neg$is_valid(x_{start}) $\vee$ $\neg$is_valid(x_{goal}) **then**
  return None, $\emptyset$
end if
$\mathcal{V} \leftarrow [x_{\text{start}}]$ ▷ Nodes list
$\mathcal{T} \leftarrow \emptyset$ ▷ Parent map
for $iter = 1$ **to** $iter_{\max}$ **do**
  $p_{\text{rand}} \leftarrow$ (rand() $<$ rate)?x_{goal} : Sample($bounds$)
  $n_{\text{near}} \leftarrow \arg\min_{n \in \mathcal{V}}$ StateDistance(n, p_{rand})
  $(n_{\text{new}}, \text{arc}) \leftarrow$ Steer$(n_{\text{near}}, p_{\text{rand}}, limits)$
  if $\neg$CollisionFree(arc) **then**
    continue
  end if
  $\mathcal{V}$.append(n_{new})
  $\mathcal{T}[n_{\text{new}}] \leftarrow (n_{\text{near}}, \text{arc})$
  if Distance$(n_{\text{new}}.xy, x_{\text{goal}}) \leq d_{\text{tol}}$ **and** CollisionFree(arc) **then**
    $\mathcal{P} \leftarrow []$
    $cur \leftarrow n_{\text{new}}$
    while $cur \neq x_{\text{start}}$ **do**
      $(p, \text{arc_p}) \leftarrow \mathcal{T}[cur]$
      $\mathcal{P}$.extend(reversed(arc_p))
      $cur \leftarrow p$
    end while
    $\mathcal{P}$.reverse()
    $\mathcal{P}$.append(x_{goal})
    return $\mathcal{P}, \mathcal{T}$
  end if
end for
return None, $\mathcal{T}$

C. Frontier-Based Exploration

To achieve full coverage, the system autonomously targets frontiers—boundaries between known free space and unknown territory.

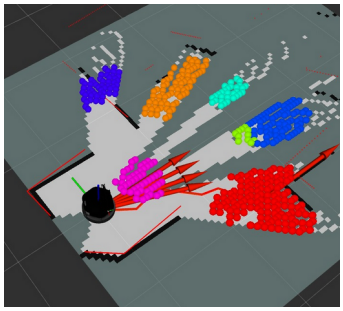


Fig. 2: Frontier detection and DBSCAN-based clustering of frontier pixels on the map boundary.

1) Detection and Clustering: The occupancy grid is processed to extract frontier pixels. DBSCAN clustering is applied with $\epsilon = 3.0$ pixels and min_samples $= 3$ to group adjacent frontier pixels into coherent target regions while preserving small but meaningful frontier segments in real maps.

2) Goal Prioritization: Each frontier cluster C_i is ranked according to a cost function:

$$\text{score}(C_i) = (\text{Size}(C_i) \cdot w_{\text{size}}) - (\text{Dist}(C_i) \cdot w_{\text{dist}}) \quad (4)$$

where $w_{\text{size}} = 1.0$ and $w_{\text{dist}} = 2.5$. This prioritizes larger, closer clusters to maximize mapping efficiency. In the real system, repeated planning failures are handled by temporary frontier-goal blacklisting (triggered by consecutive empty planned-path outputs), forcing selection of alternative frontiers instead of retrying the same narrow-passage target. Algorithm 2 details the baseline frontier identification and ranking process.

Algorithm 2 Frontier Exploration

Require: Map $\mathcal{M}$, Pose (x, y, θ), Validation function
Ensure: Goal (x_g, y_g) or None

▷ Extract frontier candidates
$F \leftarrow \{c \in \text{free} \mid \text{neighbor}(c) \in \text{unknown}\}$
$C \leftarrow \text{Cluster}(F, \epsilon = 3.0)$ via DBSCAN
▷ Evaluate and score clusters
$\mathcal{S} \leftarrow []$
for each $C_i \in C$ **do**
  $\text{m} \leftarrow \text{medoid}(C_i)$ ▷ cluster representative
  $d \leftarrow \text{dist}(\text{m}, \text{robot})$
  $\text{score} \leftarrow w_s|C_i| - w_d \cdot d$
  $\mathcal{S}.\text{add}((\text{score}, \text{m}))$
end for
▷ Select best frontier
Sort $\mathcal{S}$ by score descending
for each $(\text{score}, \text{m}) \in \mathcal{S}$ **do**
  if Valid(m) **then**
    return m
  end if
end for
return None

D. Pure Pursuit Trajectory Controller

The path follower executes RRT-planned paths using pure pursuit control. Let robot pose be (x, y, θ) and lookahead point on path be $(x_{\text{LA}}, y_{\text{LA}})$ at distance L. Define heading error:

$$\alpha = \text{atan2}(y_{\text{LA}} - y, x_{\text{LA}} - x) - \theta \quad (5)$$

The steering command at forward speed v is:

$$\omega = \frac{2v \sin(\alpha)}{L} \quad (6)$$

For small α, this provides near-linear feedback: $\omega \approx \frac{2v\alpha}{L}$. Practical safeguards include:

- Enforce $L \geq \varepsilon > 0$ and clamp $\omega \in [-\omega_{\max}, \omega_{\max}]$
- If $|\alpha| > \pi/3$, use spin-in-place: $v = 0, \omega = \text{sign}(\alpha) \cdot \omega_{\max}$
- Scale forward speed by turn severity ($s = |\alpha|/(\pi/3)$):

$$v = \max(0.2v_{\max}, v_{\max}(1 - s)) \quad (7)$$

- Apply final deceleration $v \leftarrow v \cdot d_{\text{final}}$ when $d_{\text{final}} < 1.0$ m

Tuning note: lookahead distance L trades responsiveness versus smoothness. RRT arcs pair well with pure pursuit since the controller targets points along smooth curves. Algorithm 3 presents the control law implementation.

Algorithm 3 Pure Pursuit Control

Require: Path $\mathcal{P}$, Pose (x, y, θ), Path index, $L_{\text{la}} = 0.15$ m
Ensure: Linear velocity v, Angular velocity ω

$d_{\text{final}} \leftarrow \text{Distance}((x, y), \mathcal{P}[-1])$
if $d_{\text{final}} < 0.15$ m **then**
  return $(0.0, 0.0)$ ▷ Goal reached
end if
▷ Find lookahead target
$p_{\text{target}} \leftarrow \mathcal{P}[-1]$
for $i \leftarrow$ path_index **to** $\text{len}(\mathcal{P}) - 1$ **do**
  if $\text{Distance}((x, y), \mathcal{P}[i]) \geq L_{\text{la}}$ **then**
    $p_{\text{target}} \leftarrow \mathcal{P}[i]$
    break
  end if
end for
▷ Geometric calculations and control law
$L \leftarrow \max(\text{Distance}((x, y), p_{\text{target}}), \varepsilon)$
$\alpha \leftarrow \text{atan2}(p_{\text{target}}.y - y, p_{\text{target}}.x - x) - \theta$
if $|\alpha| > \pi/3$ **then**
  return $(0.0, \text{sign}(\alpha) \cdot \omega_{\max})$
end if
$v \leftarrow \max(0.2v_{\max}, v_{\max}(1.0 - |\alpha|/(\pi/3)))$
if $d_{\text{final}} < 1.0$ m **then**
  $v \leftarrow v \cdot d_{\text{final}}$
end if
$\omega \leftarrow \text{clamp}(2v \sin(\alpha)/L, -\omega_{\max}, \omega_{\max})$
return (v, ω)

IV. Implementation & Experimental Setup

A. Overview

The system integrates three concurrent modules: a frontier detector that proposes exploration targets, a kinodynamic RRT planner that generates collision-free, dynamically feasible arc trajectories, and a Pure-Pursuit tracker that follows the RRT trajectory while monitoring path validity and triggering replans. Experiments evaluate mapping coverage, planner success, and trajectory execution quality under realistic sensor and compute constraints.

B. Hardware and Simulation Setup

- **Platforms:** Experiments run in Gazebo/Ignition and on a differential-drive robot (TurtleBot family). Both setups utilize the same ROS2 software stack to ensure high reproducibility.
- **Sensors:** A 2D LiDAR is utilized for occupancy grid mapping; high-resolution wheel encoders provide odometry tracking. SLAM Toolbox constructs the occupancy grid used by the planner.
- **Compute:** Executed on a single onboard CPU (laptop or embedded unit). CPU utilization and per-plan laten-

cies are recorded to characterize real-time computational expense.

- **ROS2 Stack:** Validated on ROS 2 Jazzy. Launch configurations are separated into simulation (`launch/sim_kino_launch.py`) and hardware deployment (`launch/real_kino_launch.py`).

C. Software Stack and Node Topology

The modular ROS2 pipeline decouples data capture, planning, and tracking via the following active nodes:

- `real_odometry_relay_node.py`: Fuses wheel-odometry twist with SLAM-consistent TF pose and publishes `/odom_opt`.
- `Frontier-Based Explorer`: Automatically extracts and filters candidate frontier clusters, publishing selected targets to `/exploration_goal`.
- `real_kino_planner.py`: Subscribes to map and odometry updates and publishes kinodynamic paths on `/planned_path`.
- `real_path_follower.py`: Tracks planned paths and publishes `geometry_msgs/TwistStamped` on `/cmd_vel`.
- `SLAM Toolbox`: Provides real-time mapping output on the `/map` topic.

Standardized topics include: `/map` (`nav_msgs/OccupancyGrid`), `/odom_opt` (`nav_msgs/Odometry`), `/exploration_goal` (`geometry_msgs/PoseStamped`), `/cmd_vel` (`geometry_msgs/TwistStamped`), and visualization markers on `/rrt_tree` and `/rrt_path_vis`. Map communication is configured with `RELIABLE` + `TRANSIENT_LOCAL` QoS.

D. Localization Variants (Real Runs)

To keep this report focused on hands-on planning, we evaluate localization only as an interchangeable upstream input to the same planning stack:

- **Version A (baseline):** SLAM Toolbox provides both map and pose-consistent TF for `/odom_opt` relay.
- **Version B (hybrid):** SLAM Toolbox remains the map source (`/map`), while robot pose for `/odom_opt` is taken from a separate GTSAM+NDT localization pipeline.

This isolates the planning contribution: frontier selection, kinodynamic planning, and tracking logic remain unchanged across both versions.

E. From Simulation to Real-World Deployment

To avoid repeating algorithmic details, this section summarizes only the practical modifications introduced beyond the simulation baseline:

- **Odometry adaptation:** Simulation uses idealized odometry frames; the real stack introduces `real_odometry_relay_node.py` to combine SLAM-aligned pose with wheel-derived twist and publish a stable `/odom_opt`.
- **Planner robustness in tight spaces:** The real planner uses heading-aware nearest-node selection and an align-then-advance steering cost so in-place rotation can be selected when forward progress is geometrically blocked.
- **Explorer anti-stuck behavior:** Real frontier exploration monitors `/planned_path`; repeated empty paths trigger temporary blacklisting of the current frontier goal and force selection of an alternative NBV.
- **Conservative safety checks:** Goal validity and path validity are both checked against occupancy with circular footprint inflation ($r = 0.20$ m), followed by replanning if invalidated by map updates.
- **Execution policy differences:** Real tracking keeps the same pure-pursuit law but applies in-place spin for large heading error and dynamic speed scaling near turns and terminal approach.

F. Map and Test Environments

The planner operates on a 2D occupancy grid from SLAM Toolbox (typical resolution 0.05 m to 0.10 m). Evaluation uses multiple environments: open rooms, linear corridors, and cluttered arenas. Operational planning bounds are extracted dynamically from the incoming occupancy grid header properties (origin coordinates, width, and height). A representative 2D occupancy grid map generated during evaluation is shown in Fig. 3.

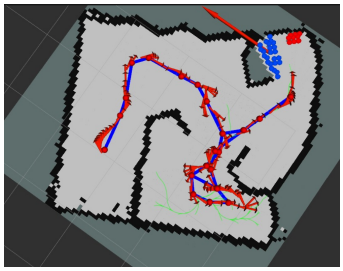


Fig. 3: 2D occupancy grid map of the evaluation environment generated by SLAM Toolbox.

G. Key Parameters

The primary configuration settings used across the software nodes are listed in Table I.

H. Implementation Notes

- The steering operation (`steer_kinodynamic`) forward-simulates a discrete set of (v, ω) primitives and evaluates them with phase-dependent cost (alignment-priority vs progress-priority), enabling in-place rotation behavior in constrained geometry.
- Collision checking is handled via fine-grained discretization along each candidate arc. Grid cells within the robot's radius are queried against the inflated map.
- RViz-based debugging is supported through publication of the tree on `/rrt_tree` and the selected final path on `/rrt_path_vis`.

TABLE I: Principal Parameters and Defaults

Parameter	Default and role
rrt_goal_sample_rate	0.10; goal bias in sampling.
rrt_max_iterations	5000; planner iteration cap.
steer_horizon Δt	0.50 s; forward-simulation horizon per expansion.
steer_num_steps	10; integration steps per steering rollout.
robot_radius	0.20 m; nominal circular robot footprint.
max_linear_vel	0.30 m/s; linear velocity limit.
max_angular_vel	1.00 rad/s; angular velocity limit.
lookahead_distance	0.15 m; Pure-Pursuit lookahead.
distance_threshold	0.15 m; goal arrival tolerance.
DBSCAN eps	3.0 pixels; clustering search radius.
DBSCAN min_samples	3; frontier cluster minimum size.
weight_size	1.0; frontier size weighting.
weight_distance	2.5; frontier distance weighting.
preemption_distance	1.0 m; distance threshold for selecting the next goal.
max_empty_paths_before_blacklist	3; consecutive empty-path limit before blocking an NBV.
blocked_goal_radius	0.50 m; radius for temporary NBV blacklist.
Occupancy collision threshold	50 map-value units; cost threshold for collision checking.
Map free cell value	0; grid value for free space.
Map unknown cell value	-1; grid value for unknown space.
replan_period	1.0 s; periodic path-validation interval.

TABLE II: Compact Simulation vs Real Differences

Parameter	Simulation	Real
Motion primitive count	15	19
robot_radius	0.25 m	0.20 m
DBSCAN min_samples	10	3

I. Experimental Protocol

To evaluate system performance consistently, $N \geq 5$ trials are executed per environment with distinct starting configurations:

1) Launch SLAM Toolbox to initialize mapping.
2) Spin up the frontier explorer to target and publish exploration goals.
3) Generate the RRT trajectory; the Pure Pursuit controller executes the path until arrival or path invalidation.
4) Continue execution loops until no further frontiers exist or a timeout is reached.
5) Record rosbags and export parameters to a local YAML file to ensure trial reproducibility.

J. Safety and Failure Handling

To avoid getting trapped in narrow passages, the real system combines conservative footprint collision checks with explicit failure recovery logic. If a planned trajectory intersects newly discovered obstacles, the planner clears and replans immediately. For large angular corrections, the controller defaults to in-place rotation before forward motion. At the exploration layer, repeated empty planned paths trigger temporary blacklisting of the current NBV and selection of alternative frontiers.

K. Reproducibility and Run Instructions

To run the setup in simulation, execute the following command:

```
ros2 launch kinodynamic_planning
       sim_kino_launch.py
```

For hardware-based deployment on the physical platform, run:

```
ros2 launch kinodynamic_planning
       real_kino_launch.py
```

L. Code and Data Availability

Including hyperlinks is recommended for reproducibility. In IEEE format, use \href{}{} or \url{} so readers can directly access your implementation and experiment data.

- **Project repository:** https://github.com/Gaafoor/Exploration-with-Kinodynamics
- **Rosbag and experiment artifacts:** Included in the same repository workspace and documented in the project README.

Rosbag files used in the experiments are included in the same repository and can be accessed through the project structure and README.

V. Results

The integrated system was evaluated in both simulation and physical environments. Quantitative metrics were gathered over multiple runs to characterize the mapping rate, kinematic motion efficiency, and trajectory tracking quality.

A. Exploration and Mapping Efficiency

The frontier-based explorer clusters raw frontier pixels into coherent target regions and prioritizes them with a size-distance score (Fig. 2). In real runs, the additional blocked-goal mechanism reduces repeated retries at geometrically constrained frontiers and improves continuity of exploration progress after planner failures.

In a representative hardware run, the system operated for 30.11 s and published 16 exploration goals with 16 frontier-cluster updates. During this interval, the planner produced 6 path messages (5 non-empty, 1 empty), with non-empty planned paths ranging from 21 to 91 waypoints (mean: 63). SLAM Toolbox provided stable map updates throughout the run, while residual unmapped areas remained concentrated in narrow corners and tight passages near the configured safety radius.

B. Planner and Control Performance

The Kinodynamic RRT planner constructs feasible arc branches (Fig. 1) under bounded linear and angular velocities with occupancy-based collision checks. The heading-aware nearest-node metric and align-first steering mode improve robustness in narrow or high-curvature sections where pure position-greedy expansion is insufficient.

For the same representative run, `/cmd_vel` was published 600 times (about 20 Hz), with measured command limits of $|v|_{\max} = 0.298$ m/s and $|\omega|_{\max} = 1.00$ rad/s, consistent with configured caps. The robot covered approximately 3.12 m (from `/odom_opt`) over a 29.26 s odometry window. Motion-state decomposition from commands shows 66.1% linear-motion time, 8.5% rotation-only time, and 25.4% stopped time. These values indicate predominantly continuous movement, with brief stops and reorientation phases tied to replanning and heading correction.

VI. Discussion

The experiments indicate that real-world reliability is governed less by nominal trajectory tracking and more by failure handling in constrained geometry. In the representative run, planner logs reported 6 successful path publications and 6 planning-failure events within the same short interval, showing that feasibility can change quickly as map information evolves. This behavior confirms the coupling between frontier selection and kinodynamic reachability: a frontier can be topologically valid yet difficult to connect under footprint inflation and bounded curvature.

A key practical trade-off is conservativeness versus reachability. Increasing safety margins improves collision robustness but reduces feasible corridors; reducing margins expands reachable space but can increase risk near walls. Similarly, stronger heading-priority steering improves escape behavior but may slightly reduce direct progress in open areas, and aggressive replanning can increase stop fractions. These trade-offs motivate parameter tuning per environment and support future work on adaptive, context-aware tuning policies.

VII. Conclusion & Future Work

This report presented a modular exploration system that combines mapping, frontier selection, kinodynamic RRT planning, and Pure Pursuit tracking. The final implementation distinguishes a simulation baseline from a deployment-oriented real stack, where odometry relay, heading-aware steering, and frontier anti-stuck handling were critical for robust operation in real environments. In addition, we documented two real localization variants for the same planning core: a SLAM Toolbox baseline and a hybrid setup that keeps SLAM Toolbox mapping while using GTSAM+NDT optimized pose for `/odom_opt`. The resulting architecture supports continuous map growth, online replanning, and stable execution under practical sensing and kinematic constraints.

Future research directions will focus on several key areas to expand the capabilities of this architecture:

- **Three-Dimensional Planning:** Extending the kinodynamic planner to support 3D costmaps and 3D motion primitives to allow deployment on unmanned aerial vehicles (UAVs) and legged platforms.
- **Dynamic Obstacle Avoidance:** Integrating local, time-elastic band or social force models into the controller loop to allow reactivity to humans and other moving agents without triggering full global replanning.
- **Multi-Robot Exploration:** Scaling the frontier exploration and prioritization heuristics to distributed, multi-agent frameworks using decentralized communications for collaborative mapping.
- **Learning-Assisted Heuristics:** Incorporating machine-learning-based estimators to predict frontier utility and clearance profiles, aiming to reduce the standard sample validation iterations required by the RRT tree under dense clutter.

Acknowledgment

The authors would like to express their sincere gratitude to Prof. Narcis Palomeras and Marta Real of the Robotics and Autonomous Systems Laboratory for their technical guidance, valuable feedback, and for providing the simulation frameworks and physical platforms that made this project possible.

References

[1] S. M. LaValle and J. J. Kuffner, "Randomized kinodynamic planning," *The International Journal of Robotics Research*, vol. 20, no. 5, pp. 378–400, 2001.

[2] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation*, 1997, pp. 146–151.

[3] S. Macenski, "SLAM Toolbox: An unified set of tools for 2D lifelong mapping," *Journal of Open Source Software*, vol. 6, no. 61, p. 2773, 2021.

[4] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Carnegie-Mellon University, Pittsburgh, PA, Tech. Rep. CMU-RI-TR-92-01, 1992.

[5] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–240.

[6] S. M. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge University Press, 2006.